

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Real-Time Sales Forecasting for a Multi-Store Retail Chain

Integrating HDFS, Kafka, PySpark, MLlib, MySQL, and Django

Prepared by group (9) members

Student ID	Names
101215	Richard Mwenedata
101216	Pascal Kabagema
101207	Denyse Mutoni
101248	Salome Mukarugaba
101188	Daniel Urimubenshi

Case Study: Sales Forecasting

Course:	Big Data Essentials
Lecturer:	Dr. Kundan Kumar
Environment:	Windows 11
Date:	September 2026

This project can be found on [GitHub](#), click this link

Table of Contents

1. Abstract	2
2. Project Overview	3
2.1 Objectives.	3
2.2 Case Study Description	3
3. System Architecture	3
3.1 Batch Training Path	4
3.2 Real-Time Streaming Path	4
3.3 Presentation Layer	4
4. Technologies Used	5
5. Dependencies	5
5.1 Python Packages	5
5.2 Java Libraries / JAR Dependencies	6
5.3 Scala Version	6
6. Step-by-Step Workflow	6
7. Findings	15
7.1 Model Performance	15
7.2 Feature Importance	15
7.3 Discussion	16
7.4 Scale Summary	17
8. Conclusion	17
9. Appendix: Project Files	17

1 Abstract

Retail organizations increasingly rely on data-driven forecasting to anticipate demand, manage inventory, and respond to real-time sales patterns. This project presents the design and implementation of an end-to-end Big Data pipeline for real-time sales forecasting, built for a simulated multi-store, multi-region retail chain, and developed as a course project for Big Data Essentials. The system integrates six core technologies. Hadoop Distributed File System (HDFS), Apache Kafka, Apache Spark (batch and Structured Streaming), Spark MLlib, MySQL, and Django into a single working architecture that spans distributed storage, real-time message streaming, distributed processing, machine learning, relational persistence, and web-based visualization, all deployed on a Windows 11 workstation.

The pipeline follows two parallel data paths that converge at a shared presentation layer. In the batch training path, a synthetically generated dataset of 17.5 million transaction records (1.12 GB), covering 25 stores across 5 regions, 200 products, and 8 categories over five years (2022-2026), is stored in HDFS. A PySpark batch job aggregates this data to daily granularity, engineers ten time-series and calendar-based features, and trains a Random Forest Regressor to forecast daily revenue, writing the resulting predictions to MySQL via JDBC. In the real-time streaming path, a Kafka producer replays the same dataset as a simulated point-of-sale feed, which a PySpark Structured Streaming consumer ingests, parses, and aggregates into live region-and-category-level totals, persisting these to MySQL through a `foreachBatch` sink. A Django web application then serves both the historical predictions and the live streaming data through four dashboard views. Overview, Predictions, Charts, and Live with the Live view polling a JSON endpoint every 2.5 seconds to reflect new data without a page reload.

The trained forecasting model was evaluated on a time-based, held-out test set of the most recent 180 days, avoiding the data leakage risk of random splitting in a time-series context. The model achieved strong predictive accuracy, with an R^2 of 0.9969, an RMSE of approximately 233,398, and an MAE of approximately 177,180 revenue units. Feature importance analysis showed that daily units sold was by far the dominant predictor (50.7% importance), followed collectively by calendar-based seasonality features month, weekend, and holiday-season indicators which together validate the deliberate demand patterns embedded in the synthetic dataset. Lag-based features, including previous-day revenue and a 7-day rolling average, contributed modestly, indicating some residual short-term momentum in the data beyond same-day attributes.

This work demonstrates that a fully integrated Big Data pipeline combining distributed storage, real-time streaming, distributed machine learning, and interactive visualization can be implemented and verified end-to-end on commodity hardware, with the live dashboard reflecting streaming updates within seconds of ingestion. The results confirm both the technical feasibility of the architecture and the interpretability of the underlying forecasting model, offering a template for similar real-time analytics systems in retail and adjacent domains.

Keywords: Big Data, Real-Time Streaming, Apache Kafka, Apache Spark, Structured Streaming, Machine Learning, Sales Forecasting, Random Forest, MySQL, Django

2 Project Overview

This project implements an end-to-end Big Data pipeline for real-time sales forecasting, developed to satisfy the Big Data Essentials course project requirements. The system integrates distributed storage, real-time message streaming, distributed data processing, machine learning, relational storage, and web-based visualization into a single working pipeline, running entirely on a Windows 11 workstation.

The chosen case study is Sales Forecasting for a simulated multi-store, multi-region retail chain. The objective is to predict daily total revenue using historical transaction patterns, seasonal effects, and real-time incoming transaction data.

2.1 Objectives

- Stream simulated real-time retail transactions using Apache Kafka, sourced from a dataset stored in HDFS.
- Process the streaming data in real time using PySpark Structured Streaming, producing both raw record views and live aggregated totals.
- Engineer time-series features from historical transaction data and train a machine learning model using Spark MLlib to forecast daily revenue.
- Persist both batch predictions and live streaming aggregates into a MySQL relational database.
- Build a Django web dashboard to visualize historical predictions, model performance, and the live data stream.

2.2 Case Study Description

The dataset simulates five years (2022-2026) of retail transactions across 25 stores in 5 regions, covering 200 products across 8 categories. Each transaction record includes the date, store, region, product, category, units sold, unit price, promotion flag, discount percentage, and resulting sales amount. Realistic patterns are embedded in the data generation process, including weekday/weekend demand variation, a November–December holiday sales spike, and promotional discount effects.

Scale 17.5M rows transactions generated	Coverage 25 / 5 / 200 stores / regions / products	Span 5 years 2022 - 2026
---	---	--

3 System Architecture

The system is composed of two parallel data paths that converge at the dashboard layer: a batch training path (historical data → trained model → stored predictions) and a real-time streaming path (live transactions → streaming aggregation → live dashboard updates). Figure 1 illustrates the full architecture.

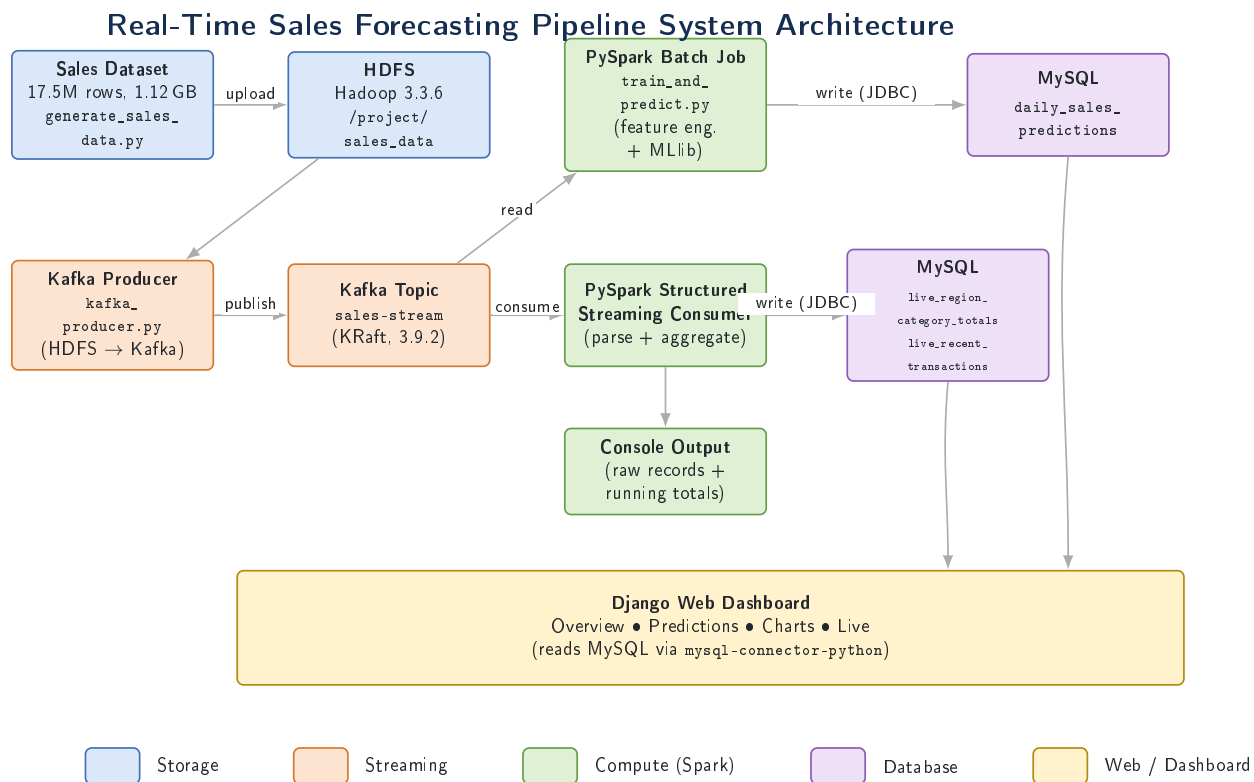


Figure 1: End-to-end system architecture

3.1 Batch Training Path

The full transaction dataset (17.5 million records, 1.12 GB) is generated locally and uploaded to HDFS. A PySpark batch job reads this dataset from HDFS, aggregates it to daily granularity, engineers time-series features, trains a Random Forest Regressor via Spark MLlib, evaluates it on a held-out time window, and writes the resulting predictions to a MySQL table via JDBC.

3.2 Real-Time Streaming Path

A Kafka producer reads the same dataset from HDFS and publishes transaction records one at a time into a Kafka topic, simulating a live point-of-sale feed at a configurable rate. A PySpark Structured Streaming job subscribes to this topic, parses each JSON message, and maintains two continuously updated views: a raw record feed and a running total of revenue and units sold grouped by region and category. Both views are printed to the console and written to MySQL via a `foreachBatch` sink, making the live data available to the web dashboard.

3.3 Presentation Layer

A Django application serves four pages, all reading exclusively from MySQL: an Overview page with summary KPIs and model accuracy metrics; a paginated Predictions table; a Charts page with four visualizations of model performance; and a Live page that polls a JSON endpoint every 2.5 seconds to reflect the real-time streaming data, including a live/idle status indicator and flash notifications when new data arrives.

4 Technologies Used

The following table lists every core technology used in the project along with the exact version installed and verified during development.

Technology	Version	Role
Java (Eclipse Temurin)	11.0.32.101	JVM runtime for Hadoop, Kafka, and Spark
Hadoop (HDFS)	3.3.6	Distributed storage for the source dataset
Apache Kafka	3.9.2 (KRaft mode)	Real-time message streaming
Apache Spark	3.5.9 (cluster) / PySpark 3.5.1	Distributed batch and stream processing, MLlib
MySQL Server	8.0 (custom port 3307)	Relational storage for predictions and live data
MySQL Connector/J	26.7.0	JDBC driver used by Spark to write to MySQL
Django	6.1	Web dashboard framework
Python	3.14.7	Primary scripting language
Chart.js	4.4.1 (via CDN)	Client-side charting in the dashboard

5 Dependencies

5.1 Python Packages

Installed in the project virtual environment.

Package	Version
pyspark	3.5.1
py4j	0.10.9.7
kafka-python	3.0.11
django	6.1
mysql-connector-python	26.7.0
pandas	3.0.5
numpy	2.5.2
scikit-learn	1.9.0
scipy	1.18.1
matplotlib	3.11.1
setuptools	latest (distutils compatibility shim for Python 3.12+)

5.2 Java Libraries / JAR Dependencies

- `winutils.exe` and `hadoop.dll` (Hadoop 3.3.6 Windows native binaries, from the community `cdarlint/winutils` repository)
- `mysql-connector-j-26.7.0.jar` added to Spark's classpath for JDBC writes to MySQL
- `spark-sql-kafka-0-10_2.12:3.5.1` and its transitive dependencies (`kafka-clients`, `spark-token-provider-kafka-0-10`, `hadoop-client-runtime`, etc.) resolved automatically via Maven through `spark.jars.packages`

5.3 Scala Version

Spark 3.5.9 (standard build) and the Kafka connector artifacts used Scala 2.12.

6 Step-by-Step Workflow

Step 1 Data Collection and Setup

A Python script (`generate_sales_data.py`) generated a synthetic retail transaction dataset of 17.5 million rows (1.12 GB), covering 25 stores, 5 regions, 200 products, and 8 categories across five years, with built-in weekday, seasonal, and promotional effects. The dataset was uploaded to HDFS at `/project/sales_data/sales_data.csv`.

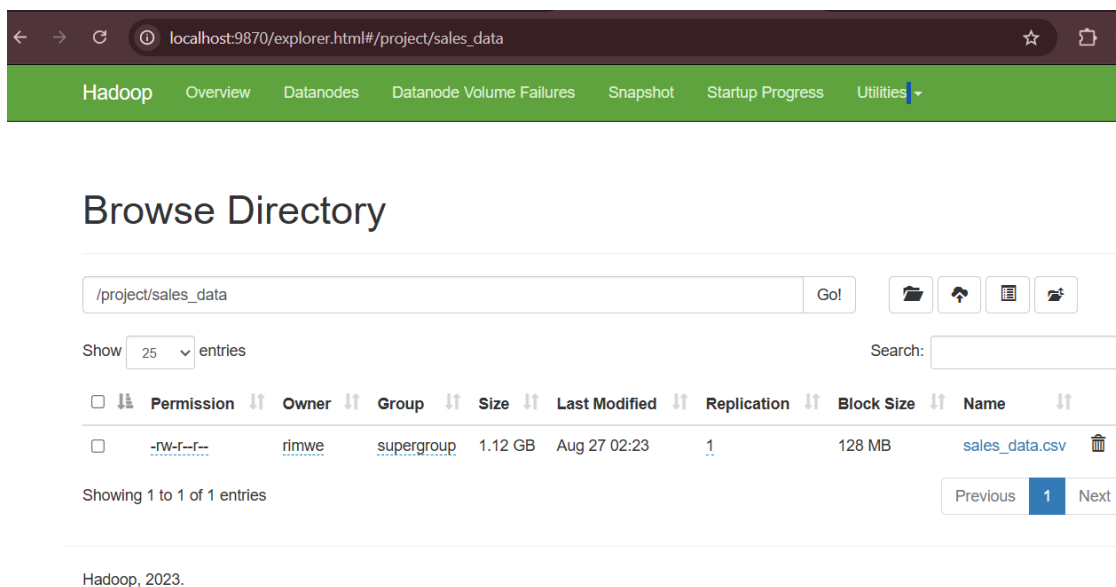


Figure 2: data loaded in hdfs

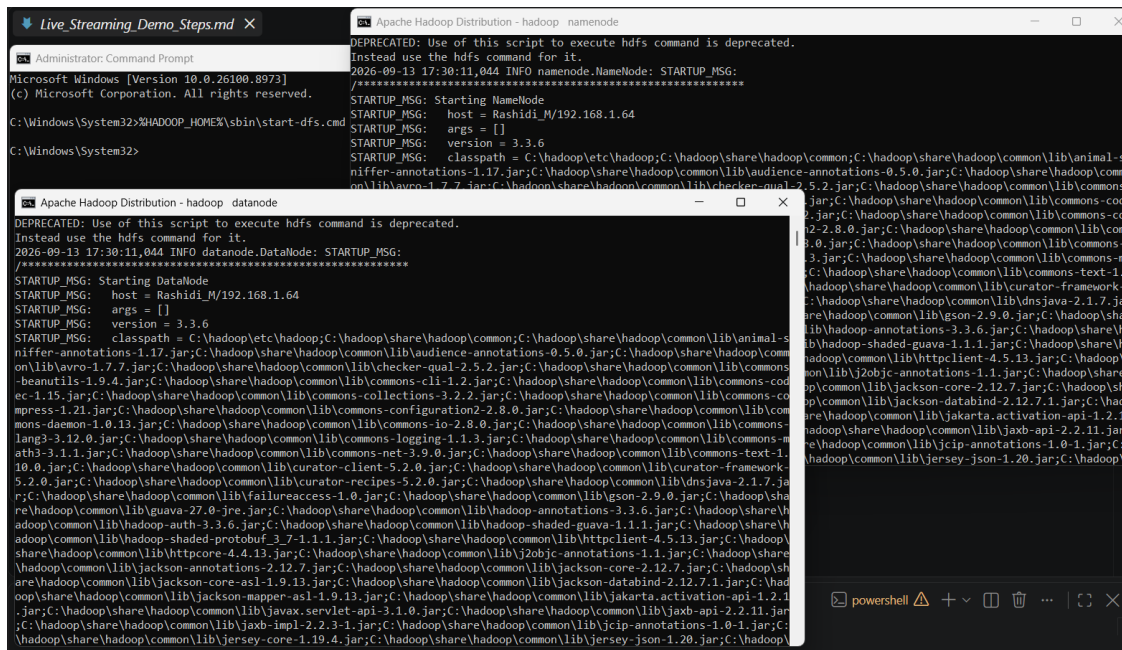


Figure 3: apache hadoop distribution

Step 2 Real-Time Data Streaming with Kafka

A Kafka producer script (`kafka_producer.py`) read the dataset from HDFS using PySpark and published each row as a JSON message to a Kafka topic (`sales-stream`) at a configurable rate, simulating live point-of-sale transactions.

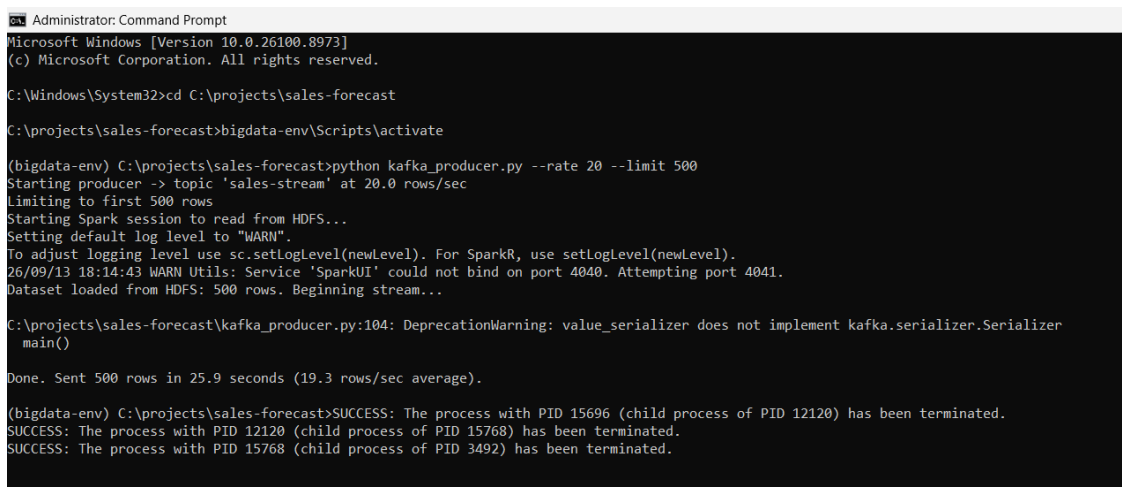


Figure 4: kafka producer


```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.26100.8973]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd C:\kafka

C:\kafka>bin\windows\kafka-topics.bat --list --bootstrap-server localhost:9092
sales-stream

C:\kafka>
```

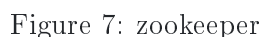
Figure 5: kafkaStreaming

```
Administrator: Command Prompt - bin\windows\kafka-server-start.bat config\kraft\server.properties
Microsoft Windows [Version 10.0.26100.8973]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd C:\kafka

C:\kafka\bin>windows\kafka-server-start.bat config\kraft\server.properties
[2026-09-13 18:09:23,636] INFO Registered kafka-type=kafka.log4jController MBean (kafka.utils.Log4jControllerRegistration$)
[2026-09-13 18:09:23,644] INFO Setting -o jdk.tls.rejectClientInitiatedRenegotiation=true to disable client-initiated TLS renegotiation (org.apache.zookeeper.common.XS909util)
[2026-09-13 18:09:23,987] INFO [ControllerServer id=1] Starting controller (kafka.server.ControllerServer)
[2026-09-13 18:09:24,336] INFO Updated connection-accept-rate max connection creation rate to 2147483647 (kafka.network.ConnectionQuotas)
[2026-09-13 18:09:24,375] INFO [SocketServer listenerType=CONTROLLER, nodeId=1] Created data-plane acceptor and processors for endpoint : listenerName(CONTROLLER) (kafka.network.SocketServer)
[2026-09-13 18:09:24,375] INFO authorizeStart completed for endpoint CONTROLLER. Endpoint is now READY. (org.apache.kafka.server.network.EndpointReadyFutures)
[2026-09-13 18:09:24,383] INFO [SharedServer id=1] Starting SharedServer (kafka.server.SharedServer)
[2026-09-13 18:09:24,431] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Recovering unflushed segment 0. 0/2 recovered for __cluster_metadata-0. (kafka.log.LogLoader)
[2026-09-13 18:09:24,431] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Loading producer state till offset 0 with message format version 2 (kafka.log.UnifiedLog$)
[2026-09-13 18:09:24,431] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Reloading from producer snapshot and rebuilding producer state from offset 0 (kafka.log.UnifiedLog$)
[2026-09-13 18:09:24,431] INFO Deleted producer state snapshot C:\tmp\kraft-combined-logs\_cluster_metadata-0\00000000000000000005002.snapshot (org.apache.kafka.storage.internals.log.SnapshotFile)
[2026-09-13 18:09:24,431] INFO Deleted producer state snapshot C:\tmp\kraft-combined-logs\_cluster_metadata-0\00000000000000000005005.snapshot (org.apache.kafka.storage.internals.log.SnapshotFile)
[2026-09-13 18:09:24,431] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Producer state recovery took 0ms for snapshot load and 0ms for segment recovery from offset 0 (kafka.log.UnifiedLog$)
[2026-09-13 18:09:24,779] INFO [ProducerStateManager partitions=__cluster_metadata-0] Wrote producer snapshot at offset 55002 with 0 producer ids in 5 ms. (org.apache.kafka.storage.internals.log.ProducerStateManager)
[2026-09-13 18:09:24,779] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Recovering unflushed segment 55002. 1/2 recovered for __cluster_metadata-0. (kafka.log.LogLoader)
[2026-09-13 18:09:24,779] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Loading producer state till offset 55002 with message format version 2 (kafka.log.UnifiedLog$)
[2026-09-13 18:09:24,779] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Reloading from producer snapshot and rebuilding producer state from offset 55002 (kafka.log.UnifiedLog$)
[2026-09-13 18:09:24,779] INFO [ProducerStateManager partitions=__cluster_metadata-0] Loading producer state from snapshot file 'SnapshotFile(offset=55002, file=C:\tmp\kraft-combined-logs\_cluster_metadata-0\00000000000000000005002.snapshot)' (org.apache.kafka.storage.internals.log.ProducerStateManager)
[2026-09-13 18:09:24,779] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Producer state recovery took 0ms for snapshot load and 0ms for segment recovery from offset 55002 (kafka.log.UnifiedLog$)
[2026-09-13 18:09:24,794] INFO [ProducerStateManager partitions=__cluster_metadata-0] Wrote producer snapshot at offset 55072 with 0 producer ids in 1 ms. (org.apache.kafka.storage.internals.log.ProducerStateManager)
[2026-09-13 18:09:24,804] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Loading producer state till offset 55072 with message format version 2 (kafka.log.UnifiedLog$)
[2026-09-13 18:09:24,808] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Reloading from producer snapshot and rebuilding producer state from offset 55072 (kafka.log.UnifiedLog$)
[2026-09-13 18:09:24,808] INFO [ProducerStateManager partitions=__cluster_metadata-0] Loading producer state from snapshot file 'SnapshotFile(offset=55072, file=C:\tmp\kraft-combined-logs\_cluster_metadata-0\00000000000000000005072.snapshot)' (org.apache.kafka.storage.internals.log.ProducerStateManager)
[2026-09-13 18:09:24,808] INFO [LogLoader partitions=__cluster_metadata-0, dir=C:\tmp\kraft-combined-logs] Producer state recovery took 0ms for snapshot load and 0ms for segment recovery from offset 55072 (kafka.log.UnifiedLog$)
[2026-09-13 18:09:24,845] INFO Initialized snapshots with IDs SortedSet(OffsetAndEpoch(offset=10184, epoch=3), OffsetAndEpoch(offset=12642, epoch=3), OffsetAndEpoch(offset=23590, epoch=7), OffsetAndEpoch(offset=26869, epoch=7), OffsetAndEpoch(offset=33193, epoch=7), OffsetAndEpoch(offset=38149, epoch=7), OffsetAndEpoch(offset=45238, epoch=7), OffsetAndEpoch(offset=52065, epoch=7)) from C:\tmp\kraft-combined-logs\_cluster_metadata-0 (kafka.raft.KafkaMetadataLog$)
[2026-09-13 18:09:24,857] INFO [raft-expiration-reaper] Starting (kafka.raft.TimingWheelExpirationService$ExpiredOperationReaper)
[2026-09-13 18:09:24,869] INFO [RaftManager id=1] Reading KRaft snapshot and log as part of the initialization (org.apache.kafka.raft.KafkaRaftClient)
[2026-09-13 18:09:24,873] INFO [RaftManager id=1] Loading snapshot (OffsetAndEpoch(offset=52065, epoch=7)) since log start offset (0) is greater than the internal listener's next offset (-1) (org.apache.kafka.raft.internals.KRaftControlRecordStateMachine)
[2026-09-13 18:09:24,914] INFO [RaftManager id=1] Starting voters are VoterSet(voters=[VoterNode(voterKey=ReplicaKey(id=1, directoryId=Optional.empty), listeners=Endpoints(endpoints=(listenerName(CONTROLLER)-1, localhost/127.0.0.1:9093))), supportedRaftVersions=SupportedVersionRange(min_version=0, max_version=0)]) (org.apache.kafka.raft.KafkaRaftClient)
[2026-09-13 18:09:24,914] INFO [RaftManager id=1] Starting request manager with static voters: [localhost:9093 (id: 1 rack: null)] (org.apache.kafka.raft.KafkaRaftClient)
[2026-09-13 18:09:25,016] INFO [RaftManager id=1] Attempting durable transition to ResignedState(localId=1, epoch=9, voters=[1], electionTimeoutMs=1037, unackedVoters=[], preferredSuccessors=[]) from null (org.apache.kafka.raft.QuorumState)
```

Figure 6: kafka start



A PySpark Structured Streaming consumer (`spark_streaming_consumer.py`) subscribed to the Kafka topic, parsed incoming JSON messages against a defined schema, and computed two live views: raw incoming records and running totals of revenue and units sold grouped by region and category.



Step 4 Prediction Modeling with MLlib

A separate batch script (**train_and_predict.py**) read the full historical dataset from HDFS, aggregated it to daily revenue totals, engineered ten features (day of week, month, weekend flag, holiday-season flag, promotion rate, average discount, transaction count, units sold, previous-day revenue, and a 7-day rolling average revenue), and trained a Random Forest Regressor using a time-based 1,645/180-day train/test split.

```
Administrator Command Prompt - python spark_streaming_consumer.py
Microsoft Windows [Version 10.0.26100.897]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd C:\projects\sales-forecast

C:\projects\sales-forecast>bigdata-env\Scripts\activate

(bigdata-env) C:\projects\sales-forecast>python spark_streaming_consumer.py
Starting Spark session (this will download the Kafka connector on first run)...
  loading settings :: url = jar:file:/C:/spark/jars/ivy-2.5.3-jar!org/apache/ivy/core/settings/ivysettings.xml
ivy Default Cache set to: C:\Users\rimel\ivy2\cache
The jars for the packages stored in: C:\Users\rimel\ivy2\jars
org.apache.spark:spark-sql-kafka-0-10-2.12 added as a dependency
:: resolving dependencies :: org.apache.spark:spark-submit-parent-b3d470e0-4e95-4bd7-a90d-3687cbcae388;1.0
confs: [default]
found org.apache.spark:spark-sql-kafka-0-10-2.12;3.5.1 in central
found org.apache.spark:spark-token-provider-kafka-0-10-2.12;3.5.1 in central
found org.apache.kafka:kafka-clients;3.4.1 in central
found org.lz4:lz4-java;1.8.0 in central
found org.xerial.snappy:snappy-java;1.1.10.3 in central
found org.slf4j:slf4j-api;2.0.7 in central
found org.apache.hadoop:hadoop-client-runtime;3.3.4 in central
found org.apache.hadoop:hadoop-client-api;3.3.4 in central
found commons-logging:commons-logging;1.3 in central
found com.google.code.findbugs:jsr305;3.0.0 in central
found org.apache.commons:commons-pool2;2.11.1 in central
:: resolution report :: resolve 40ms :: artifacts dl 10ms
:: modules in use:
com.google.code.findbugs:jsr305;3.0.0 from central in [default]
commons-logging:commons-logging;1.3 from central in [default]
org.apache.commons:commons-pool2;2.11.1 from central in [default]
org.apache.hadoop:hadoop-client-api;3.3.4 from central in [default]
org.apache.hadoop:hadoop-client-runtime;3.3.4 from central in [default]
org.apache.kafka:kafka-clients;3.4.1 from central in [default]
org.apache.spark:spark-sql-kafka-0-10-2.12;3.5.1 from central in [default]
org.apache.spark:spark-token-provider-kafka-0-10-2.12;3.5.1 from central in [default]
org.lz4:lz4-java;1.8.0 from central in [default]
org.slf4j:slf4j-api;2.0.7 from central in [default]
org.xerial.snappy:snappy-java;1.1.10.3 from central in [default]
:: Conflicts in 0:
::=
| conf | modules | | artifacts | | | |
|---|---|---|---|---|---|---|
| default | 11 | 0 | 0 | 0 | 11 | 0 |
:: retrieving :: org.apache.spark:spark-submit-parent-b3d470e0-4e95-4bd7-a90d-3687cbcae388
confs: [default]
0 artifacts copied, 11 already retrieved (0kB/21ms)
Setting default log level to "WARN".
```

Figure 9: pyspark streaming consumer

Step 5 Store Data and Predictions in MySQL

Model predictions on the held-out test period were written to a MySQL table (`daily_sales_predictions`) via Spark's JDBC writer. The streaming consumer separately wrote live aggregates (`live_region_category_totals`) and a rolling transaction feed (`live_recent_transactions`) to MySQL using a `foreachBatch` sink.

```
mysql> use sales_forecast;
Database changed
mysql> show tables;
+-----+
| Tables_in_sales_forecast |
+-----+
| daily_sales_predictions  |
| live_recent_transactions |
| live_region_category_totals |
+-----+
3 rows in set (0.00 sec)
```

Figure 10: Database and tables

```
mysql> select * from daily_sales_predictions
-> LIMIT 20;
```

date	actual_revenue	predicted_revenue	day_of_week_num	is_weekend	is_holiday_season
2026-07-05	14861451.860000001	15167813.129086863	1	1	0
2026-07-06	10508451.560000002	10367076.066624826	2	0	0
2026-07-07	10355020.590000002	10261729.404504903	3	0	0
2026-07-08	10388737.33	10364629.515661133	4	0	0
2026-07-09	11266449.720000003	11201648.367605291	5	0	0
2026-07-10	14623797.769999996	14304865.105793513	6	0	0
2026-07-11	17776444.45	17439594.740356077	7	1	0
2026-07-12	15380160.389999997	15302905.651387477	1	1	0
2026-07-13	10609156.090000004	10381358.707689721	2	0	0
2026-07-14	10161283.959999999	10129991.60592963	3	0	0
2026-07-15	10254518.920000002	10305759.049288249	4	0	0
2026-07-16	11240946.180000003	11234484.142734312	5	0	0
2026-07-17	13870776.21	14162393.424773805	6	0	0
2026-07-18	18067405.209999993	17760711.309512824	7	1	0
2026-07-19	14901792.69	15314775.594979221	1	1	0
2026-07-20	10376409.709999999	10539115.471725732	2	0	0
2026-07-21	10016257.409999996	10279372.691710046	3	0	0
2026-07-22	10254831.879999995	10245928.233807227	4	0	0
2026-07-23	11246197.51	11203202.77599935	5	0	0
2026-07-24	14123442.8	14161820.60720042	6	0	0

```
20 rows in set (0.00 sec)

mysql> SELECT COUNT(*) FROM daily_sales_predictions;
+-----+
| COUNT(*) |
+-----+
|      180 |
+-----+
1 row in set (0.00 sec)
```

Figure 11: daily sales predictions

```
mysql> SELECT * FROM live_region_category_totals
-> LIMIT 20;
```

region	category	total_revenue	total_units_sold	updated_at
West	Electronics	955453.78	2538	2026-08-27 11:48:14
South	Sports	389746.3100000002	2769	2026-08-27 11:48:14
Central	Electronics	725681.4899999996	1960	2026-08-27 11:48:14
North	Home	391239.81000000035	3258	2026-08-27 11:48:14
North	Sports	373616.4299999998	2655	2026-08-27 11:48:14
East	Electronics	1067561.87	2978	2026-08-27 11:48:14
Central	Home	368584.56999999983	2798	2026-08-27 11:48:14
West	Sports	373192.26000000053	2673	2026-08-27 11:48:14
South	Electronics	1117899.2499999995	2616	2026-08-27 11:48:14
North	Electronics	1145360.6799999995	2890	2026-08-27 11:48:14
South	Home	441273.20000000007	2895	2026-08-27 11:48:14
West	Home	394137.82	2589	2026-08-27 11:48:14
West	Beauty	151107.25999999998	2632	2026-08-27 11:48:14
East	Apparel	118572.98000000004	2421	2026-08-27 11:48:14
Central	Beauty	117994.22999999988	2246	2026-08-27 11:48:14
South	Beauty	144623.68999999994	2986	2026-08-27 11:48:14
South	Toys	117822.02999999999	2712	2026-08-27 11:48:14
Central	Apparel	139047.32999999996	3018	2026-08-27 11:48:14
North	Beauty	125287.43999999994	2626	2026-08-27 11:48:14
South	Apparel	152921.65000000005	2891	2026-08-27 11:48:14

```
20 rows in set (0.00 sec)

mysql> SELECT COUNT(*) FROM live_region_category_totals;
+-----+
| COUNT(*) |
+-----+
|      80 |
+-----+
1 row in set (0.00 sec)
```

Figure 12: live region category totals

```
mysql> SELECT * FROM live_region_category_totals
-> LIMIT 20;
```

region	category	total_revenue	total_units_sold	updated_at
West	Electronics	955453.78	2538	2026-08-27 11:48:14
South	Sports	389746.3100000002	2769	2026-08-27 11:48:14
Central	Electronics	725681.4899999996	1960	2026-08-27 11:48:14
North	Home	391239.81000000035	3258	2026-08-27 11:48:14
North	Sports	373616.4299999998	2655	2026-08-27 11:48:14
East	Electronics	1067561.87	2978	2026-08-27 11:48:14
Central	Home	368584.56999999983	2798	2026-08-27 11:48:14
West	Sports	373192.26000000053	2673	2026-08-27 11:48:14
South	Electronics	1117899.2499999995	2616	2026-08-27 11:48:14
North	Electronics	1145360.6799999995	2890	2026-08-27 11:48:14
South	Home	441273.20000000007	2895	2026-08-27 11:48:14
West	Home	394137.82	2589	2026-08-27 11:48:14
West	Beauty	151107.25999999998	2632	2026-08-27 11:48:14
East	Apparel	118572.98000000004	2421	2026-08-27 11:48:14
Central	Beauty	117994.22999999988	2246	2026-08-27 11:48:14
South	Beauty	144623.68999999994	2986	2026-08-27 11:48:14
South	Toys	117822.02999999999	2712	2026-08-27 11:48:14
Central	Apparel	139047.32999999996	3018	2026-08-27 11:48:14
North	Beauty	125287.43999999994	2626	2026-08-27 11:48:14
South	Apparel	152921.65000000005	2891	2026-08-27 11:48:14

```
20 rows in set (0.00 sec)

mysql> SELECT COUNT(*) FROM live_region_category_totals;
```

COUNT(*)
80

```
1 row in set (0.00 sec)
```

Figure 13: live region category totals

Step 6 Build a Django Web Dashboard

A Django application was built with four pages Overview, Predictions, Charts, and Live all reading from the MySQL database using the `mysql-connector-python` Django backend. The Live page polls a JSON API endpoint every 2.5 seconds to reflect new streaming data without a page reload.

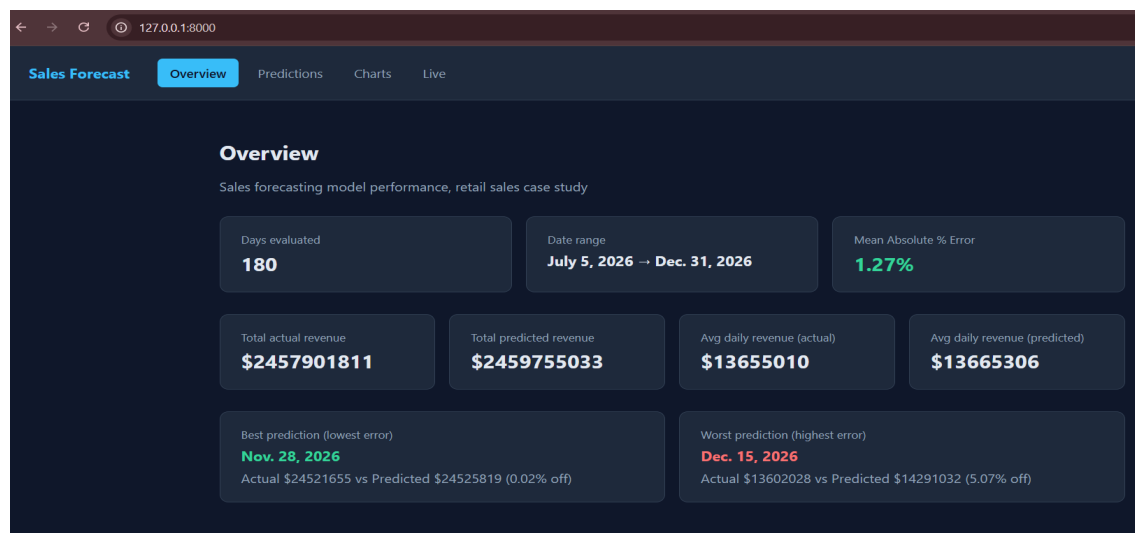


Figure 14: Django webdashboard

DATE	ACTUAL REVENUE	PREDICTED REVENUE	ERROR	% ERROR	FLAGS
Dec. 31, 2026	\$15935288.39	\$15892956.25	-42332.14	0.27%	Holiday season
Dec. 30, 2026	\$14991590.41	\$14832988.65	-158601.76	1.06%	Holiday season
Dec. 29, 2026	\$14266574.05	\$14273557.93	6983.88	0.05%	Holiday season
Dec. 28, 2026	\$14715606.74	\$14684790.77	-30815.97	0.21%	Holiday season
Dec. 27, 2026	\$21769941.52	\$21878465.88	108524.36	0.50%	Weekend Holiday season
Dec. 26, 2026	\$24374257.50	\$24606740.55	232483.05	0.95%	Weekend Holiday season
Dec. 25, 2026	\$19677339.64	\$19561588.06	-115751.58	0.59%	Holiday season
Dec. 24, 2026	\$15818223.28	\$15812777.92	-5445.36	0.03%	Holiday season
Dec. 23, 2026	\$15017126.06	\$14924389.44	-92736.62	0.62%	Holiday season
Dec. 22, 2026	\$14154187.78	\$14255802.36	101614.58	0.72%	Holiday season
Dec. 21, 2026	\$14871457.07	\$15044892.11	173435.04	1.17%	Holiday season

Figure 15: predictions on the web page

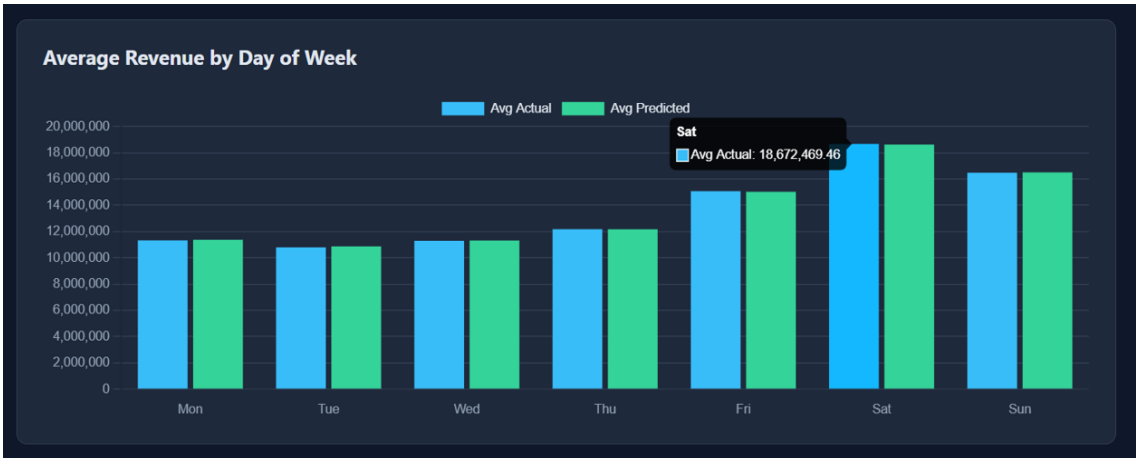


Figure 16: average revenue by week

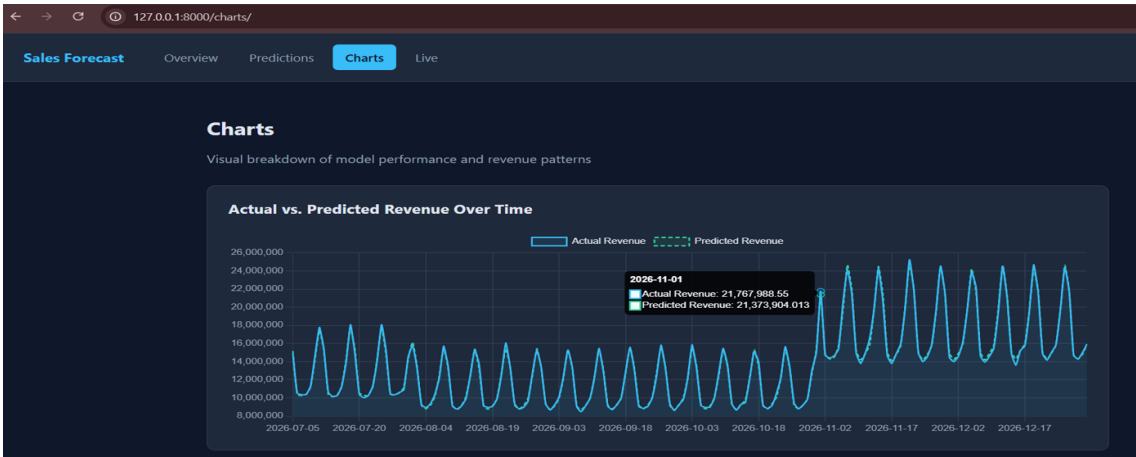


Figure 17: actual vs predicted revenue

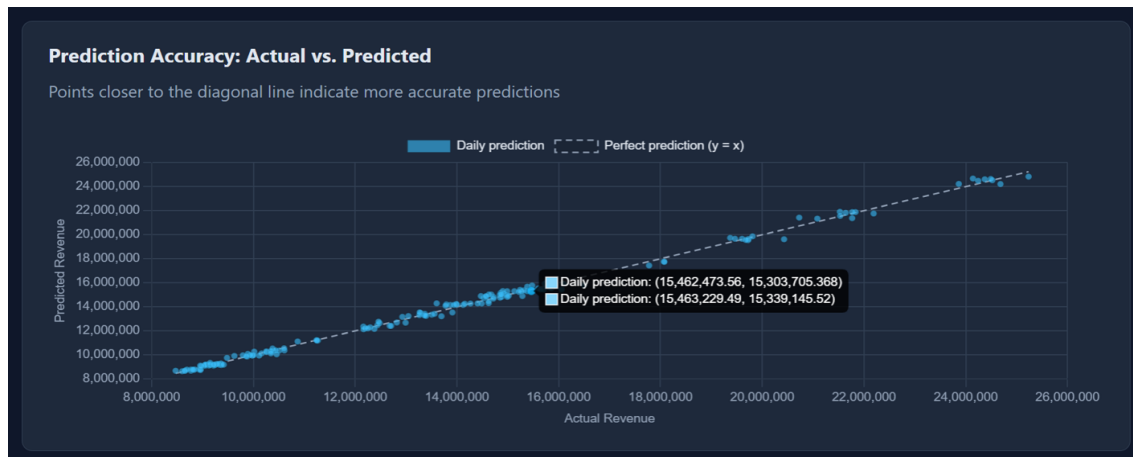


Figure 18: prediction accuracy

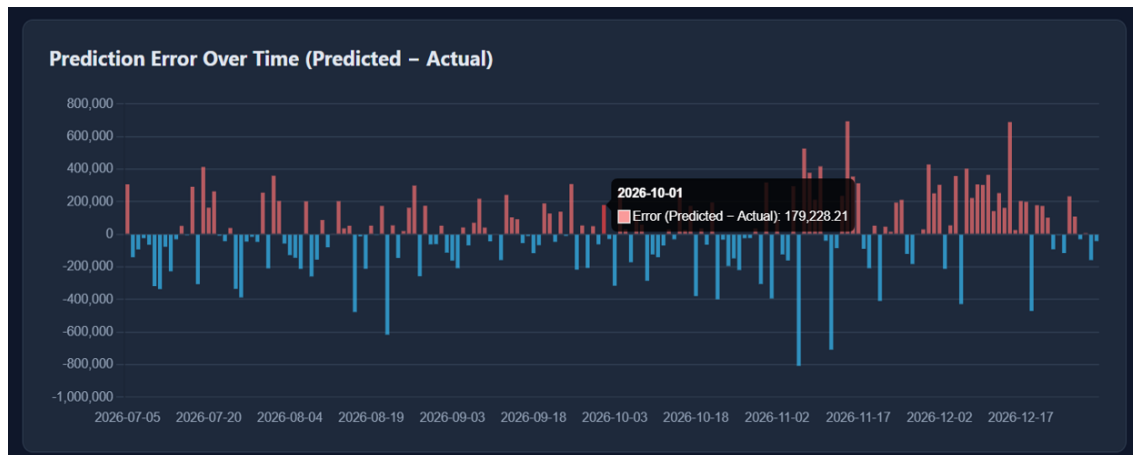


Figure 19: prediction error

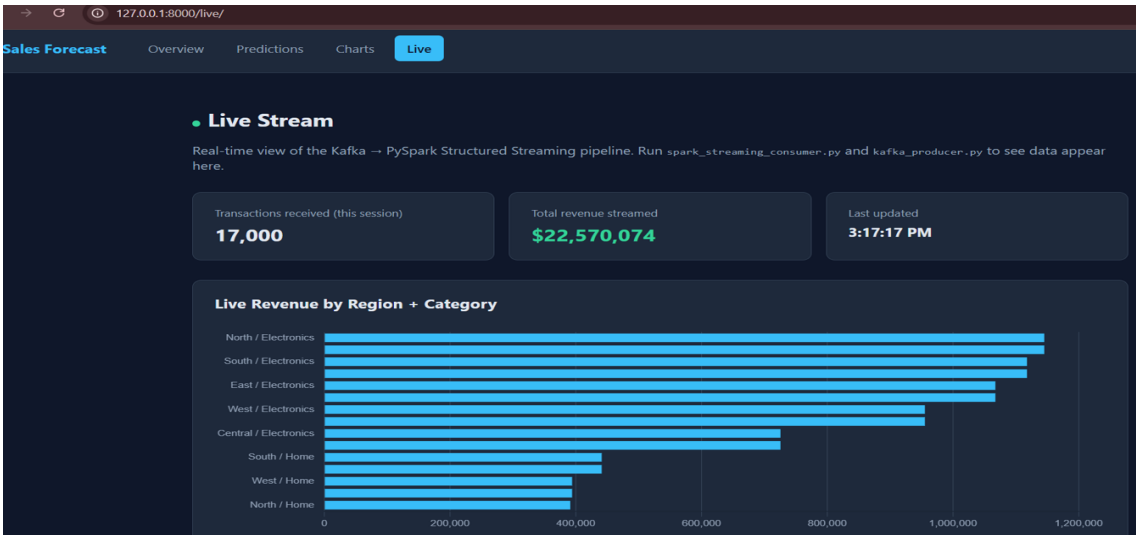


Figure 20: Live streaming dashboard

Step 7 Testing and Deployment

Each pipeline component was verified independently (HDFS file listing, Kafka topic creation and message delivery, Spark shell startup, MySQL connectivity) before being connected end to end. The full pipeline was run together and verified via a live demonstration: starting HDFS, Kafka, and MySQL, running the streaming consumer and producer concurrently, and confirming that the Django dashboard reflected new data within seconds.

7 Findings

7.1 Model Performance

The Random Forest Regressor (100 trees, maximum depth 8) was evaluated on a held-out set of the most recent 180 days of daily revenue, following a time-based (not random) train/test split appropriate for forecasting tasks.

Metric	Value	Interpretation
RMSE	233,398.14	Average magnitude of prediction error, in revenue currency units
MAE	177,179.90	Average absolute prediction error
R ²	0.9969	The model explains 99.69% of the variance in daily revenue

7.2 Feature Importance

The trained model’s feature importances reveal which factors most strongly drive daily revenue predictions:

Feature	Importance
daily_units_sold	0.5067
month_num	0.1168
is_weekend	0.1168
is_holiday_season	0.1044
day_of_week_num	0.0803
prev_day_revenue	0.0488
rolling_7day_avg_revenue	0.0224
transaction_count	0.0022
avg_discount_pct	0.0010
promo_rate	0.0008

7.3 Discussion

- Units sold is by far the dominant predictor (50.7% importance), which is expected since revenue is mechanically driven by volume this confirms the model has correctly learned the core relationship in the data.
- Calendar-based seasonality (month, weekend, and holiday-season flags) collectively account for roughly a third of the model's predictive power, validating the deliberate weekday and holiday-season effects built into the synthetic dataset.
- The lag-based features (previous-day revenue and the 7-day rolling average) contribute modestly but meaningfully, indicating short-term revenue momentum carries some forecasting value beyond same-day attributes alone.
- Promotion-related fields (**promo_rate**, **avg_discount_pct**) show minimal standalone importance, likely because their effect is already captured indirectly through **daily_units_sold**, since promotions in the dataset primarily act by boosting the units-sold figure.
- The very high R^2 (0.9969) reflects both a genuinely learnable underlying pattern and the fact that the daily aggregation smooths out much of the transaction-level noise present in the raw 17.5-million-row dataset.

7.4 Scale Summary

Item	Value
Raw transaction records	17,500,000 rows
Raw dataset size in HDFS	1.12 GB
Aggregated daily records	1,825 days (2022-2026)
Training set	1,645 days
Test set (held out)	180 days (most recent)
Kafka topic partitions	1 (single-node development setup)

8 Conclusion

This project successfully demonstrates a complete, working Big Data pipeline that spans distributed storage, real-time streaming, distributed processing, machine learning, relational persistence, and web-based visualization. Every component HDFS, Kafka, PySpark (both batch and Structured Streaming), Spark MLlib, MySQL, and Django was integrated and verified end to end on a Windows development environment, including a live-updating dashboard that reflects the real-time streaming pipeline within seconds. The trained Random Forest model achieved strong predictive accuracy ($R^2 = 0.9969$) on held-out data, and its feature importances offer genuine, interpretable insight into the drivers of daily retail revenue.

9 Appendix: Project Files

File	Purpose
<code>generate_sales_data.py</code>	Synthetic dataset generator
<code>kafka_producer.py</code>	Reads HDFS data and streams it into Kafka
<code>spark_streaming_consumer.py</code>	PySpark Structured Streaming consumer; writes live data to MySQL
<code>train_and_predict.py</code>	Batch feature engineering, MLlib model training, and prediction storage
<code>sales_dashboard/</code> (Django project)	Web dashboard: Overview, Predictions, Charts, Live pages